

8,31

Practica 1 – Robots

Pregunta #1: ¿Cuál es la versión y el nombre de la distribución de ROS instalada en el contenedor que se le ha proporcionado? ¿Cuándo finaliza su soporte? ¿Por qué?

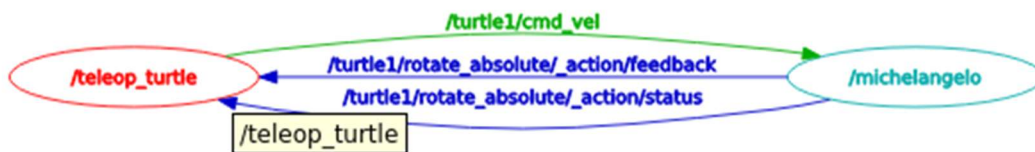
La versión es la número 2 y la distribución es humble (hawksbill). El soporte finaliza en mayo de 2027 porque es una versión LTS (Long Term Support) de ROS 2 y está alineado con Ubuntu 22.04 LTS (Jammy Jellyfish).

Pregunta #2: ¿Qué otro comando se puede ejecutar con ros2 node? Escriba ros2 node-h para obtener ayuda.

Se puede ejecutar `ros2 node info /<node_name>`. En nuestro caso, `ros2 node info /turtlesim` ¿Qué hace?

Pregunta #3: ¿Cómo se llama el tema (topic) donde /teleop_turtle publica los comandos de movimiento? Adjunte una captura de pantalla de rqt_graph donde se vea el nombre.

Se llama `/turtle1/cmd_vel`



Pregunta #4: ¿Qué recibe /michelangelo de /teleop_turtle?

Recibe un vector **de tipo** linear y otro de tipo angular.

`ros2 topic list -t` linear y angular son los nombres. El tipo es Vector3.

`/turtle1/cmd_vel [geometry_msgs/msg/Twist]`

Físicamente, ¿qué es lo que recibe? La velocidad lineal y la angular a la que debe moverse.

El tipo de mensaje es `geometry_msgs/msg/Twist`.

```

❖ ros@humble:/workspaces/sim_ws$ ros2 topic echo /turtle1/cmd_vel
linear:
  x: -2.0
  y: 0.0
  z: 0.0
angular:
  x: 0.0
  y: 0.0
  z: 0.0
---
```

Pregunta #5: ¿Qué tipos de datos se intercambian entre el nodo de teleoperación y TurtleSim?

```

ros@humble:/workspaces/sim_ws$ ros2 interface show geometry_msgs/msg/Twist
# This expresses velocity in free space broken into its linear and angular parts.

Vector3 linear
  float64 x
  float64 y
  float64 z
Vector3 angular
  float64 x
  float64 y
  float64 z

```

Pregunta #6: ¿Qué campos hay que completar en package.xml? ¿Qué es exactamente una licencia en este contexto? ¿Puede citar algunas licencias de software?

Los campos que hay que completar son *name*, *version*, *description*, *maintainer* y *license*.

```

<?xml version="1.0"?>
<?xml-model href="http://download.ros.org/schema/package_format3.xsd" schematypens="http://www.w3.org/2001/XMLSchema-instance" type="package_format3"/>
<package format="3">
  <name>tutorial_pubsub_py</name>
  <version>0.0.0</version>
  <description>TODO: Package description</description>
  <maintainer email="202313967@alu.comillas.edu">ros</maintainer>
  <license>TODO: License declaration</license>

  <test_depend>ament_copyright</test_depend>
  <test_depend>ament_flake8</test_depend>
  <test_depend>ament_pep257</test_depend>
  <test_depend>python3-pytest</test_depend>

```

Algunas posibles licencias son:

Apache-2.0, BSL-1.0, BSD-2.0, BSD-2-Clause, BSD-3-Clause, GPL-3.0-only, LGPL-3.0-only, MIT, MIT-0

Pregunta #7: ¿Qué hay que añadir a setup.py?

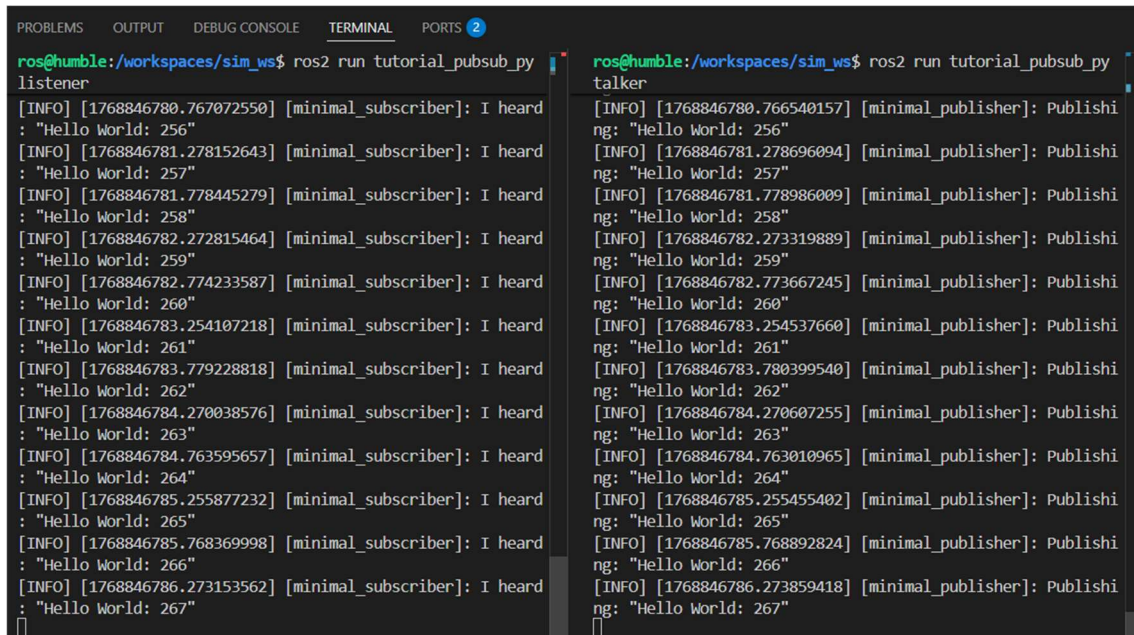
Hay que rellenar los argumentos *maintainer*, *maintainer_email*, *description* and *license*, con los valores que hemos puesto en nuestro archivo package.xml .

Y lo más importante... ¡los puntos de entrada (*entry_points*)!

Pregunta #8 : ¿Cuáles son los pasos para construir y ejecutar un nodo?

Primero creamos el nodo dentro de un paquete, luego creamos la configuración, después compilamos para poder actualizar los nuevos cambios que hemos hecho en el paquete. Después de todos estos pasos ya reconoce el nodo creado y está listo para su ejecución.

La idea de esta pregunta es que os prepararais una lista de pasos a seguir (que incluya los comandos) cuando haya que hacerlo en el futuro.



```

ros@humble:/workspaces/sim_ws$ ros2 run tutorial_pubsub_py listener
[INFO] [1768846780.767072550] [minimal_subscriber]: I heard : "Hello World: 256"
[INFO] [1768846781.278152643] [minimal_subscriber]: I heard : "Hello World: 257"
[INFO] [1768846781.778445279] [minimal_subscriber]: I heard : "Hello World: 258"
[INFO] [1768846782.272815464] [minimal_subscriber]: I heard : "Hello World: 259"
[INFO] [1768846782.774233587] [minimal_subscriber]: I heard : "Hello World: 260"
[INFO] [1768846783.254107218] [minimal_subscriber]: I heard : "Hello World: 261"
[INFO] [1768846783.779228818] [minimal_subscriber]: I heard : "Hello World: 262"
[INFO] [1768846784.270038576] [minimal_subscriber]: I heard : "Hello World: 263"
[INFO] [1768846784.763595657] [minimal_subscriber]: I heard : "Hello World: 264"
[INFO] [1768846785.255877232] [minimal_subscriber]: I heard : "Hello World: 265"
[INFO] [1768846785.768369998] [minimal_subscriber]: I heard : "Hello World: 266"
[INFO] [1768846786.273153562] [minimal_subscriber]: I heard : "Hello World: 267"

ros@humble:/workspaces/sim_ws$ ros2 run tutorial_pubsub_py talker
[INFO] [1768846780.766540157] [minimal_publisher]: Publishing: "Hello World: 256"
[INFO] [1768846781.278696094] [minimal_publisher]: Publishing: "Hello World: 257"
[INFO] [1768846781.778986009] [minimal_publisher]: Publishing: "Hello World: 258"
[INFO] [1768846782.273319889] [minimal_publisher]: Publishing: "Hello World: 259"
[INFO] [1768846782.773667245] [minimal_publisher]: Publishing: "Hello World: 260"
[INFO] [1768846783.254537660] [minimal_publisher]: Publishing: "Hello World: 261"
[INFO] [1768846783.780399540] [minimal_publisher]: Publishing: "Hello World: 262"
[INFO] [1768846784.270607255] [minimal_publisher]: Publishing: "Hello World: 263"
[INFO] [1768846784.763010965] [minimal_publisher]: Publishing: "Hello World: 264"
[INFO] [1768846785.255455402] [minimal_publisher]: Publishing: "Hello World: 265"
[INFO] [1768846785.768892824] [minimal_publisher]: Publishing: "Hello World: 266"
[INFO] [1768846786.273859418] [minimal_publisher]: Publishing: "Hello World: 267"

```

Pregunta #9: En sus propias palabras, explique brevemente cómo se comunica el nodo parlante (talker) con el oyente (listener).

Publica un mensaje en el topic llamado topic donde hay un nodo suscrito que escucha/recibe el mensaje. El mensaje es de tipo String y es un “Hello World” con un int que incrementa cada 0.5 segundos.

```

ros@humble:/workspaces/sim_ws$ ros2 topic list
/parameter_events
/rosout
/topic
ros@humble:/workspaces/sim_ws$ ros2 topic info /topic
Type: std_msgs/msg/String
Publisher count: 1
Subscription count: 1
ros@humble:/workspaces/sim_ws$

```

Pregunta #10 (): Utilice las herramientas de terminal de ROS 2 y rqt para descubrir los nodos que operan el TurtleBot3, así como los publicadores y suscriptores que tienen. Explore también la estructura y el contenido de los mensajes e intente explicar con sus palabras para qué sirve cada nodo a partir de esa información. Ignore todos los temas (topics) que empiecen con el prefijo rcl_.

```

turtlebot@imat-tb3-022:~$ ros2 node list
/diff_drive_controller
/ld08_driver
/robot_state_publisher
/turtlebot3_node

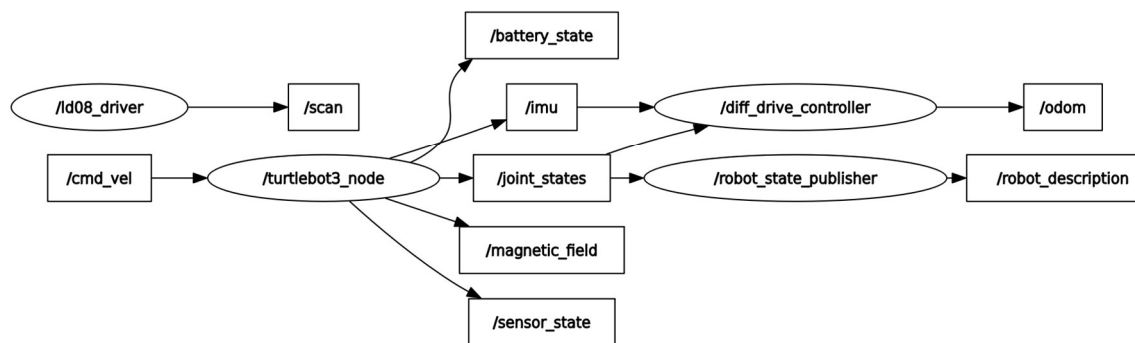
```

```

turtlebot@imat-tb3-022:~$ ros2 topic list -v
Published topics:
* /battery_state [sensor_msgs/msg/BatteryState] 1 publisher
* /imu [sensor_msgs/msg/Imu] 1 publisher
* /joint_states [sensor_msgs/msg/JointState] 1 publisher
* /magnetic_field [sensor_msgs/msg/MagneticField] 1 publisher
* /odom [nav_msgs/msg/Odometry] 1 publisher
* /parameter_events [rcl_interfaces/msg/ParameterEvent] 6 publishers
* /robot_description [std_msgs/msg/String] 1 publisher
* /rosout [rcl_interfaces/msg/Log] 6 publishers
* /scan [sensor_msgs/msg/LaserScan] 1 publisher
* /sensor_state [turtlebot3_msgs/msg/SensorState] 1 publisher
* /tf [tf2_msgs/msg/TFMessage] 2 publishers
* /tf_static [tf2_msgs/msg/TFMessage] 1 publisher

Subscribed topics:
* /cmd_vel [geometry_msgs/msg/Twist] 1 subscriber
* /imu [sensor_msgs/msg/Imu] 1 subscriber
* /joint_states [sensor_msgs/msg/JointState] 2 subscribers
* /parameter_events [rcl_interfaces/msg/ParameterEvent] 6 subscribers

```



El nodo **/ld08_driver** es el sensor LiDAR que publica sus medidas al topic **/scan** de tipo `sensor_msgs/msg/LaserScan` visto en clase. El **/cmd_vel** es un topic que publica un mensaje tipo `Twist`. El nodo **/turtlebot3_node** es como el “cerebro” del TurtleBot3 a nivel de hardware/sensores: se suscribe a **/cmd_vel** (`Twist`) para recibir las velocidades lineal y angular que le queremos mandar al robot (más adelante en la práctica), y publica varios topics con el estado y sensores del robot:

- **/battery_state** (`sensor_msgs/BatteryState`): información de batería (porcentaje/voltaje/estado).
- **/imu** (`sensor_msgs/Imu`): orientación y aceleraciones/giros del robot.
- **/joint_states** (`sensor_msgs/JointState`): estado de las “articulaciones”, normalmente ruedas (posición/velocidad).
- **/magnetic_field** (`sensor_msgs/MagneticField`): campo magnético (brújula).
- **/sensor_state** (`turtlebot3_msgs/SensorState`): estado de sensores internos del TurtleBot3 (un resumen/estado propio del robot).

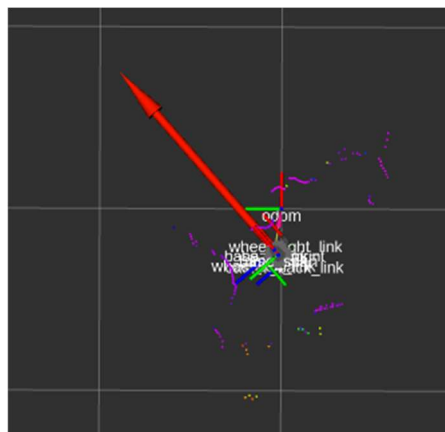
El nodo `/diff_drive_controller` se encarga del control de movimiento tipo “differential drive” (dos ruedas). Por el grafo se ve que recibe información (por ejemplo `/imu`) y publica `/odom` (`nav_msgs/Odometry`), que es la odometría: una estimación de la posición y orientación del robot a lo largo del tiempo (`x`, `y`, `yaw` + velocidades estimadas).

El nodo `/robot_state_publisher` usa la descripción del robot y los `/joint_states` para calcular y publicar la relación entre frames (transformaciones) del robot (las TF). En el grafo se ve claro que `/joint_states` $\rightarrow$ `/robot_state_publisher` $\rightarrow$ `/robot_description` (`std_msgs/String`), o sea: toma el estado de las ruedas/articulaciones y lo conecta con el modelo del robot para tener su estado “geométrico” consistente.

Pregunta #11 (): ¿Qué criterio de signos utiliza el TurtleBot3 para la velocidad angular ω por defecto? ¿Coincide con el habitual?

El TurtleBot3 utiliza los ejes: `x`, `y` de manera normal pero la `z` utiliza la convención de signos al revés, como un dron. Por lo que no coincide con el habitual (hacia abajo es positivo). Esto hace que donde nosotros nos pudiéramos esperar una velocidad angular positiva (hacia la izquierda), el TurtleBot3 la interprete en el sentido contrario (hacia la derecha). Todo este es causado por la inversión de la coordenada `z` frente al sistema de referencia usual.

Cabe destacar que comparando con la asignatura de sistemas dinámicos la referencia a la parte frontal y posterior del robot están invertidas.



1. (0,5 puntos) Siga el tutorial del siguiente enlace hasta el punto 2 inclusive para crear un mensaje personalizado, pero utilice `amr_msgs` como nombre del paquete en lugar del que sugiere el ejemplo. Para la estructura interna del mensaje, quizá lo más sencillo sea utilizar un `string`, dado que eso es lo que devuelve la librería `sshkeyboard`, pero siéntase libre de codificar los datos de otra manera.

Hemos seguido el tutorial y hemos creado un mensaje `Keyboard.msg` de tipo `string`, con la variable llamada “`key`”. Hemos modificado los archivos `package.xml` y `CMakeLists.txt`.

2. (0,75 puntos) Programe un archivo keyboard_node.py que contenga un nodo encargado de publicar cada pulsación detectada por sshkeyboard —tendrá que consultar la documentación de la librería— utilizando el mensaje personalizado anterior.

```
from sshkeyboard import listen_keyboard
import rclpy
from rclpy.node import Node
from std_msgs.msg import String
from amr_msgs.msg import Keyboard
# El nodo keyboard va a publicar el mensaje personalizado del apartado
anterior. Publicará la tecla del teclado que se presione
class MinimalPublisher(Node): Los nombres deberían ser más descriptivos.
    def __init__(self):
        super().__init__("minimal_publisher")
        # Creamos el publicador de las teclas, usando el mensaje Keyboard
        creado previamente
        self.publisher_ = self.create_publisher(Keyboard, "key", 10)
        # Usamos el metodo de listen_keyboard de sshkeyboard y cuando se
        presione cualquier tecla
        # tenemos un callback a nuestra función gestora
        listen_keyboard(on_press=self.press)

    # Funcion gestora que publica el mensaje
    def press(self, key):
        msg=Keyboard()
        msg.key=key
        self.publisher_.publish(msg)
        self.get_logger().info(f"Publishing: key={msg.key}") # Para poder
        debuggear que tecla enviamos

#Instanciamos el nodo y realizamos el metodo spin para que siempre esté
viendo si se pulsa alguna tecla
def main(args=None):
    rclpy.init(args=args)
    minimal_publisher = MinimalPublisher()
    rclpy.spin(minimal_publisher)
    minimal_publisher.destroy_node()
    rclpy.try_shutdown()

if __name__ == "__main__":
    main()
```

No solo el código tiene que ir en inglés.
También los comentarios.

-0,5

3. (1,5 puntos) En teleoperation_node.py, codifique un segundo nodo que se suscriba a las pulsaciones del teclado, modifique las consignas de velocidad lineal y angular, y las publique en el tema por el que el TurtleBot3 espera recibirlas. Puede utilizar tantas teclas como quiera para tener mayor manejo del robot según las circunstancias. Por seguridad, convendría que al menos hubiera una tecla (¿la barra espaciadora por ser la más grande?) que permitiera detener el robot de inmediato.

```
import rclpy
from rclpy.node import Node
from amr_msgs.msg import Keyboard
from geometry_msgs.msg import Twist
from sensor_msgs.msg import LaserScan
from rclpy.qos import QoSProfile, QoSReliabilityPolicy, QoSHistoryPolicy,
QoSDurabilityPolicy
import math

# El nodo teleoperation se va a suscribir al nodo keyboard pero a la vez
va a publicar la velocidad
class MinimalPubSub(Node): De nuevo, hay nombres mejores.
    def __init__(self)-> None:
        super().__init__("minimal_pub_sub")

        # Nos suscribimos al nodo Keyboard
        self._key_subscriber = self.create_subscription(
            msg_type=Keyboard,
            topic="key",
            callback=self._key_listened_callback,
            qos_profile=10
        )
        qos = QoSProfile(
            reliability=QoSReliabilityPolicy.BEST_EFFORT,
            durability=QoSDurabilityPolicy.VOLATILE,
            history=QoSHistoryPolicy.KEEP_LAST,
            depth=10
        )

        # Nos suscribimos al LiDAR para poder ver las distancias
        self._lidar_subscriber = self.create_subscription(
            msg_type=LaserScan,
            topic="scan",
            callback=self._lidar_callback,
            qos_profile=qos
        )

        # Creamos el publicador de las velocidades del robot
        self.publisher_ = self.create_publisher(Twist, "cmd_vel", 10)
```

```

# Almacenamos en una variable la anterior velocidad
self._last_vels = [0.0, 0.0]

def _key_listened_callback(self,msg:Keyboard):
    new_msg=Twist()
    new_msg.linear.x = self._last_vels[0]
    new_msg.angular.z = self._last_vels[1]
    # Cada vez que pulsamos una tecla actualizamos la velocidad como
corresponde
    if msg.key == "space":
        new_msg.linear.x = 0.0
        new_msg.angular.z = 0.0
    elif msg.key == "w":
        new_msg.linear.x += 0.05
    elif msg.key == "a":
        new_msg.angular.z -= 0.2
    elif msg.key == "s":
        new_msg.linear.x -= 0.05
    elif msg.key == "d":
        new_msg.angular.z += 0.2
    else:
        return
    # Modificamos la velocidad del robot
    self.publisher_.publish(new_msg)
    self._last_vels=[new_msg.linear.x,new_msg.angular.z]
    # Mostramos por terminal la velocidad del robot
    self.get_logger().info(f"Publishing: vels={self._last_vels}")

def _lidar_callback(self,msg:LaserScan):
    new_msg=Twist()
    new_msg.linear.x = self._last_vels[0]
    new_msg.angular.z = self._last_vels[1]
    n = len(msg.ranges)
    # El primer scan será el del rayo que mira hacia delante
    d0 = self._clean(msg.ranges[0])
    # El scan situado aproximadamente en el medio de la lista será el
del rayo que mira hacia atrás
    dhalf = self._clean(msg.ranges[n // 2])
    self.get_logger().info(
        f"n={n} | ranges[0]={d0:.3f} m | ranges[n//2]={dhalf:.3f} m"
    )
    blocked = False
    threshold= 0.3
    # Si la distancia del rayo hacia delante es menor que el threshold
sólo permitimos que avance hacia atrás
    if d0 < threshold and new_msg.linear.x > 0.0:

```



```

    new_msg.linear.x = 0.0
    blocked = True

    # Si la distancia del rayo hacia atrás es menor que el threshold
    sólo permitimos que avance hacia delante
    if dhalf < threshold and new_msg.linear.x < 0.0:
        new_msg.linear.x = 0.0
        blocked = True

    # Actualizamos la velocidad solo si se ha encontrado con un
    obstaculo
    if blocked:
        self.publisher_.publish(new_msg)
        self._last_vels = [new_msg.linear.x, new_msg.angular.z]

def _clean(self, x: float) -> float:
    # Aparecen muchos NaNs y valores que necesitamos limpiar
    if x is None or math.isnan(x) or math.isinf(x) or x <= 0.0:
        return float('inf')
    return x

def main(args=None):
    # Inicializamos el nodo
    rclpy.init(args=args)
    minimal_pub_sub = MinimalPubSub()
    rclpy.spin(minimal_pub_sub)
    minimal_pub_sub.destroy_node()
    rclpy.try_shutdown()

if __name__ == '__main__':
    main()

```

4. (0,75 puntos) Como aún está aprendiendo a conducir el TurtleBot3, vamos a ponerle unos ruedines a su nodo de teleoperación para prevenir accidentes. Suscríbase al tema que publica el LiDAR y utilice los haces que estime oportunos para supervisar que los comandos que se generan desde el teclado no van a provocar una colisión. No olvide comprobar el perfil de calidad de servicio (QoS).

Para prevenir colisiones, el nodo de teleoperación se suscribe al topic /scan, que publica el LiDAR, y supervisa que los comandos enviados desde el teclado no provoquen choques. Esto puede verse directamente en el código de arriba, donde el nodo recibe mensajes LaserScan y actúa en consecuencia antes de publicar velocidades.

Lo importante y distinto en este apartado es el perfil de Calidad de Servicio (QoS) usado al suscribirse a /scan, que la elegimos luego de ver el QoS real del publicador, tal y como se observa con el comando:

ros2 topic info /scan -v

```

turtlebot@imat-tb3-022:~/tb3_ws$ ros2 topic info /scan -v
Type: sensor_msgs/msg/LaserScan

Publisher count: 1

Node name: ld08_driver
Node namespace: /
Topic type: sensor_msgs/msg/LaserScan
Endpoint type: PUBLISHER
GID: 01.0f.a7.94.e0.13.84.92.01.00.00.00.00.00.12.03.00.00.00.00.00.00.00
QoS profile:
  Reliability: BEST_EFFORT
  History (Depth): UNKNOWN
  Durability: VOLATILE
  Lifespan: Infinite
  Deadline: Infinite
  Liveliness: AUTOMATIC
  Liveliness lease duration: Infinite

```

De esa salida observamos los siguientes campos relevantes del QoS:

- **Reliability: BEST_EFFORT**
Indica que el sistema no garantiza la entrega de todos los mensajes. Si se pierde algún escaneo del LiDAR, no se reintenta enviarlo. Esto es adecuado para sensores de alta frecuencia como el LiDAR, donde es preferible recibir datos nuevos rápidamente en tiempo real antes que asegurar cada mensaje.
- **Durability: VOLATILE**
Significa que solo se reciben mensajes publicados después de que el suscriptor se conecta. No se almacenan mensajes antiguos. Queremos las medidas actuales del LiDAR para detectar colisiones, no medidas antiguas.
- **History(Depth)**
El historial indica cuántos mensajes se almacenan para el suscriptor. En el código se usa `KEEP_LAST` con profundidad 10, suficiente para mantener algunos mensajes recientes sin acumular información vieja que ya no es relevante para evitar colisiones.
- **Lifespan, Deadline y Liveliness**
Aparecen como `Infinite / Automatic`, lo que implica que no hay restricciones temporales estrictas ni supervisión avanzada de latidos. Esto es normal en un sensor como el LiDAR, donde simplemente se procesan los datos a medida que llegan.

En resumen, el QoS utilizado en la suscripción a `/scan` replica el del publicador (`BEST_EFFORT` y `VOLATILE`), asegurando compatibilidad y un comportamiento adecuado para datos de sensores en tiempo real.